

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Tharankeswaran M	Reg. No.:	192424081
Date:	29/08/26	Duration:	60 minutes

Code of Conduct

I Tharankeswaran M (192424081) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tharankeswaran M

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4

TOTAL	/25
-------	-----

1 Requirement and Pipeline Analysis

1.1 Assigned Problem Statement

The assigned problem is the design and implementation of a **Smart Sports Stadium Operations Platform**. The system monitors crowd movement, optimizes entry and exit flows, coordinates security operations, manages parking availability, tracks concession performance, and generates event analytics. Expected outcomes include improved spectator safety, reduced waiting times, better crowd management, enhanced event experience, and more efficient stadium operations.

1.2 Project Characteristics Relevant to CI/CD

- Multi-component architecture: Python/FastAPI backend, React frontend, PostgreSQL database, Redis cache.
- Safety-critical nature: Real-time crowd density and security features demand high reliability and rapid, controlled releases.
- Multiple environments required: Development, staging (pre-event testing), and production (live stadium operations).
- Frequent updates expected during event seasons (new analytics features, security patches, performance improvements).
- Containerized deployment using Docker and Docker Compose (or Kubernetes in production).

1.3 Identified CI/CD Needs

1. Automated integration of code changes from multiple developers working on backend, frontend, and infrastructure.
2. Consistent and reproducible builds of backend and frontend artifacts.
3. Comprehensive automated testing (unit, integration, API, and basic end-to-end) on every change.
4. Static code quality and security analysis to catch defects early.
5. Automated packaging into Docker images.
6. Controlled promotion of builds through development → staging → production environments.
7. Fast feedback to developers and operations teams via notifications.
8. Reliable rollback capability in case a deployment introduces issues during a live event.

These requirements drive the design of a robust, multi-stage CI/CD pipeline that balances speed of delivery with the reliability demanded by a safety-sensitive stadium operations system.

2 Pipeline Architecture and Workflow

2.1 Overall Pipeline Stages

The CI/CD pipeline consists of the following sequential stages:

1. **Source Code Management:** Code is committed and pushed to a Git repository. Branch protection rules enforce pull-request reviews.
2. **Trigger:** Pipeline is triggered on push to `develop/main` or on pull-request creation/update.
3. **Build:** Compile/transpile frontend assets and prepare backend Python environment; build Docker images.
4. **Unit & Integration Testing:** Execute automated test suites.
5. **Code Quality & Security Analysis:** Run static analysis, linting, and vulnerability scanning.
6. **Packaging:** Tag and push Docker images to a container registry.
7. **Deploy to Staging:** Automatically deploy successful builds from the `develop` branch to the staging environment.
8. **Manual Approval / Smoke Tests:** Optional human gate before production.
9. **Deploy to Production:** Controlled deployment to the live stadium environment (often timed around event windows).
10. **Post-Deployment Verification & Monitoring:** Health checks, notifications, and readiness for rollback.

2.2 Detailed Workflow from Commit to Deployment

1. Developer creates a feature branch, implements changes, and opens a pull request targeting `develop`.
2. CI pipeline runs automatically: build, unit tests, integration tests, linting, and security scans.
3. If all checks pass and code review is approved, the PR is merged into `develop`.
4. Merge to `develop` triggers a full pipeline that builds Docker images, pushes them to the registry, and deploys to the staging environment.
5. Staging is validated (automated smoke tests + optional manual verification).
6. When ready for release, a release branch or tag is created from `develop`. After final checks, the build is promoted to production via a controlled deployment job (often requiring manual approval).
7. Production deployment updates the live services. Health checks confirm success; monitoring and alerting remain active.

8. In case of failure, an automated or one-click rollback restores the previous stable version.

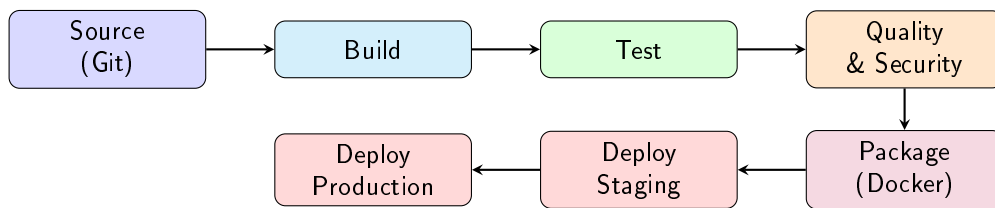


Figure 1: High-level CI/CD Pipeline Stages

3 Tools and Technology Selection

Stage	Tool	Justification
Source Control	Git + GitHub / GitLab	Industry standard; excellent branching, PR workflows, and integration with CI platforms.
CI/CD Orchestration	GitHub Actions or GitLab CI	Native integration with the repository, free tier sufficient for student/project use, YAML-based configuration, strong ecosystem of actions.
Build	Docker + multi-stage Dockerfiles	Reproducible builds; consistent artifacts across environments; already used in the project.
Unit Testing	pytest (backend), Jest / React Testing Library (frontend)	Widely adopted, excellent assertion libraries, easy CI integration.
Integration / API Testing	pytest + FastAPI TestClient, Postman/Newman or pytest-httpx	Validates service interactions and API contracts.
Code Quality	flake8 / black / isort (Python), ESLint / Prettier (JS)	Enforces consistent style and catches common errors early.
Security Scanning	Bandit, Safety (Python), npm audit / Snyk, Trivy (containers)	Detects known vulnerabilities in dependencies and container images.
Container Registry	GitHub Container Registry or Docker Hub	Seamless integration with the chosen CI platform.
Deployment	Docker Compose (staging), Kubernetes or Docker Swarm (production)	Matches the project's containerization strategy; supports rolling updates and scaling.
Notifications	Slack / Email / GitHub notifications	Immediate feedback to developers and operations staff.
Monitoring	Prometheus + Grafana (or cloud equivalents)	Real-time visibility into application health after deployment.

Table 1: Selected Tools and Justification for Each Pipeline Stage

The chosen toolchain prioritizes open-source or freely available tools that integrate well with each other and align with the existing Docker-based architecture of the Smart Stadium platform.

4 Automated Testing and Quality Integration

4.1 Testing Strategy

- **Unit Tests:** Cover core business logic (crowd density calculation, parking allocation algorithms, security alert rules, concession metrics). Target high coverage of pure functions and services.
- **Integration Tests:** Verify interactions between API layer, database, and Redis. Use testcontainers or a dedicated test database.
- **API / Contract Tests:** Ensure endpoints return expected status codes and payloads for both success and error cases.
- **Frontend Component Tests:** Validate key React components render correctly and respond to user interactions.
- **Smoke / Health Checks:** Lightweight post-deployment tests that confirm critical endpoints are reachable and return healthy status.
- **Regression Suite:** Run the full suite on every merge to `develop` and before production promotion.

4.2 Quality and Security Gates

- Linting and formatting must pass (non-zero exit code fails the pipeline).
- Static security analysis (Bandit, dependency vulnerability scanners) must report no high/critical issues.
- Container image scanning (Trivy) must pass before images are pushed to the registry.
- Minimum code coverage threshold (e.g., 70–80% for critical modules) can be enforced.

All of the above checks are executed automatically inside the CI pipeline. A failing quality or security gate blocks the merge or deployment, ensuring only validated code progresses.

5 Deployment, Security and Rollback Strategy

5.1 Environment Design

- **Development:** Local developer machines and short-lived feature environments. Docker Compose is used for rapid iteration.
- **Staging / Pre-production:** Mirrors production topology as closely as possible. Used for full integration testing, performance checks, and operator training before live events.
- **Production:** Live stadium environment. Deployments are carefully timed (preferably outside peak event hours) and require explicit approval.

5.2 Deployment Process

1. Successful pipeline on `develop` automatically deploys the new images to staging.
2. Automated smoke tests run against staging.
3. A release candidate is created (tag or release branch).
4. Production deployment job is triggered (manual approval gate recommended for safety-critical systems).
5. Rolling update or blue-green strategy is used so that there is minimal downtime.
6. Post-deployment health checks verify service availability.

5.3 Security Practices

- Secrets (database credentials, API keys) are stored in the CI platform’s secret store or an external vault; never committed to the repository.
- Least-privilege service accounts for deployment.
- Container images are scanned for vulnerabilities before promotion.
- Branch protection and required reviews on `main` and `develop`.
- Network policies and TLS encryption in production.

5.4 Rollback Mechanism

- Every successful production deployment records the previous image tags / Kubernetes deployment revision.
- A dedicated “rollback” pipeline job (or one-click action) redeploys the last known-good version.
- Database migrations are designed to be backward-compatible where possible, or reversible scripts are prepared.
- Monitoring alerts trigger immediate investigation; rollback can be initiated within minutes if critical issues appear.

This combination of automated promotion, manual gates for production, continuous monitoring, and rapid rollback provides the reliability required for a stadium operations platform.

6 Pipeline Diagram and Workflow Explanation

6.1 Complete Pipeline Flow

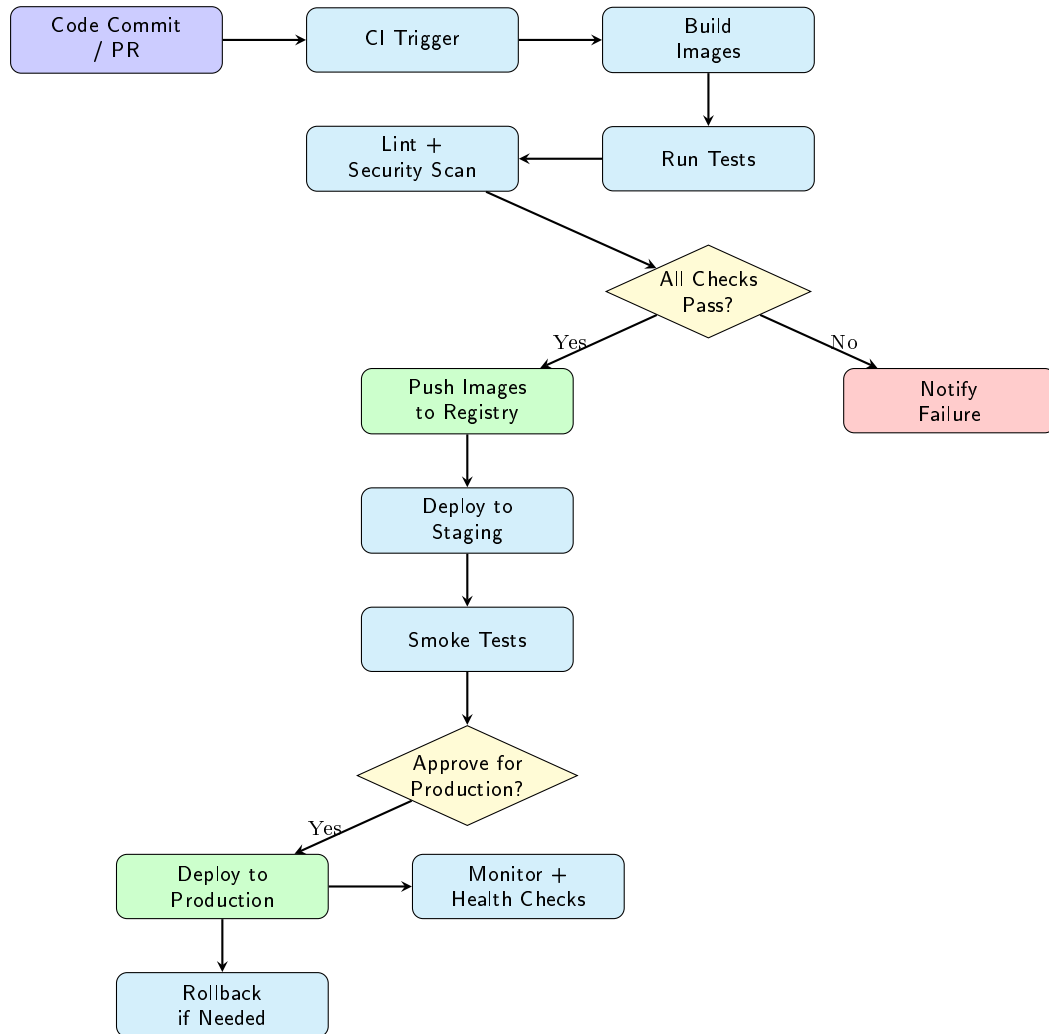


Figure 2: Detailed CI/CD Pipeline Workflow from Commit to Production

6.2 Explanation of the Workflow

1. A developer pushes code or opens a pull request. The CI system is automatically triggered.
2. The pipeline builds the application (frontend assets and backend) and constructs Docker images using multi-stage Dockerfiles.
3. Unit, integration, and API tests are executed. Concurrently or sequentially, linting and security scanners run.
4. If any test or quality gate fails, the pipeline stops and notifies the team (email/Slack-/GitHub status). The developer must fix the issues and re-trigger.
5. On success, Docker images are tagged and pushed to the container registry.

6. The new images are deployed to the staging environment. Automated smoke tests verify basic functionality.
7. For production, a manual approval step is recommended given the safety-critical nature of the platform. Once approved, the production deployment proceeds using a rolling or blue-green strategy.
8. Post-deployment health checks and monitoring confirm the system is operating correctly. If problems are detected, the rollback mechanism restores the previous stable version quickly.

This end-to-end automated flow significantly reduces manual effort, shortens feedback cycles, and increases confidence in every release of the Smart Sports Stadium Operations Platform.

7 Conclusion

The designed CI/CD pipeline fully addresses the needs of the Smart Sports Stadium Operations Platform. It provides continuous integration of code changes, automated building and testing, rigorous quality and security checks, controlled progression through environments, and reliable deployment with rollback capability. By selecting appropriate open-source tools that integrate with the existing Docker-based architecture, the pipeline is both practical to implement and aligned with industry best practices for a multi-component, safety-sensitive system.